

La désaturation des roues de réaction par utilisation de magnéto-coupleurs pour des satellites en orbite basse

VIVIAN Tom
SCEI : 45640

TIPE 2024-2025

Introduction : contrôle d'attitude



Figure 1 – image représentant un satellite

Sommaire

Roue de réaction et magnéto-coupleur

Modélisation du satellite et du champ magnétique

Résolution du problème

Application : nanosatellite CubeSat

1 – Roue de réaction et magnéto-coupleur

Principe des roues de réaction

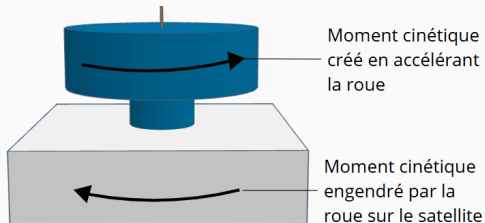
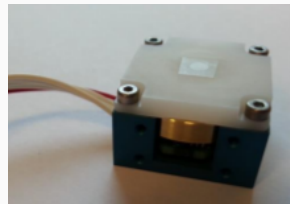


Figure 2 – modèle de roue de réaction sur son support



**Figure 3 – roue de réaction HT-RW200
(25x25x15mm³)**

Action de la roue sur le satellite

$\mathcal{R}$ référentiel lié au satellite

$$[\vec{L}_G(\text{roue})]_{\mathcal{R}} = I_{rz} \cdot \vec{\omega}_r$$

$$[\vec{L}_G(\text{sat})]_{\mathcal{R}} = I_{sz} \cdot \vec{\omega}_s$$

et

$$\vec{L}_G(\text{total}) = \vec{L}_G(\text{sat}) + \vec{L}_G(\text{roue})$$

Via le TMC dans $\mathcal{R}$:

$$\left[\frac{d\vec{L}_G}{dt}(\text{total}) \right]_{\mathcal{R}} = \sum \vec{M}_{\text{ext}}^G = \vec{0}$$

$$\Rightarrow I_{rz}\dot{\omega}_r = -I_{sz}\dot{\omega}_s$$

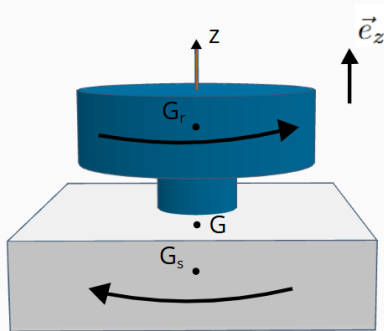


Figure 4 - modèle de roue de réaction sur son support

Magnéto-coupleur : origine du moment magnétique

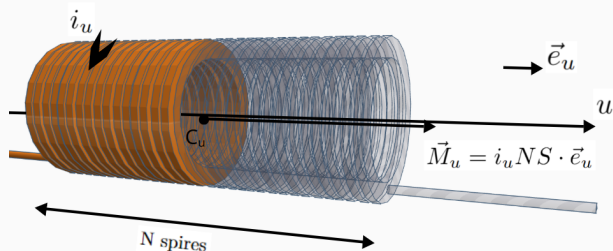


Figure 5 – moment magnétique d'un solénoïde parcouru par un courant i



Figure 6 – magnéto-coupleur HT-MTQ400 (65x16x16mm³)

Interaction avec le champ magnétique

Avec $\vec{B}_{ext}$ un champ uniforme :

$$\vec{\Gamma}_u = \vec{M}_u \times \vec{B}_{ext}$$

$$\vec{\Gamma}_u = [\vec{\Gamma}_G(\vec{M}_u \rightarrow \text{sat})]_{\mathcal{R}}$$

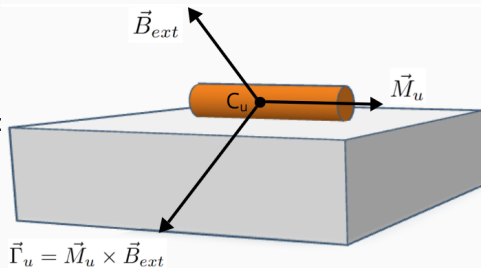


Figure 7 - modèle de magnéto-coupleur sur un support

Quels moments magnétiques les magnéto-coupleurs doivent-ils créer pour désaturer une roue de réaction sans modifier l'attitude générale du satellite par rapport à la Terre ?

Désaturer : réduire suffisamment la vitesse de rotation de la roue jusqu'à atteindre une vitesse appropriée

2 – Satellite et champ magnétique

Modélisation du satellite

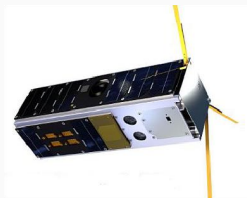


Figure 8 – nanosatellite Aalto-1 ($34 \times 10 \times 10 \text{ cm}^3$)

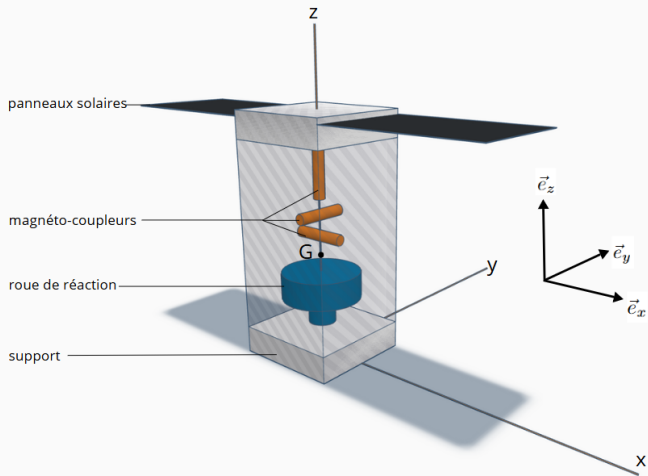


Figure 9 – modèle d'un nanosatellite CubeSat ($30 \times 10 \times 10 \text{ cm}^3$)

Modélisation de l'orbite

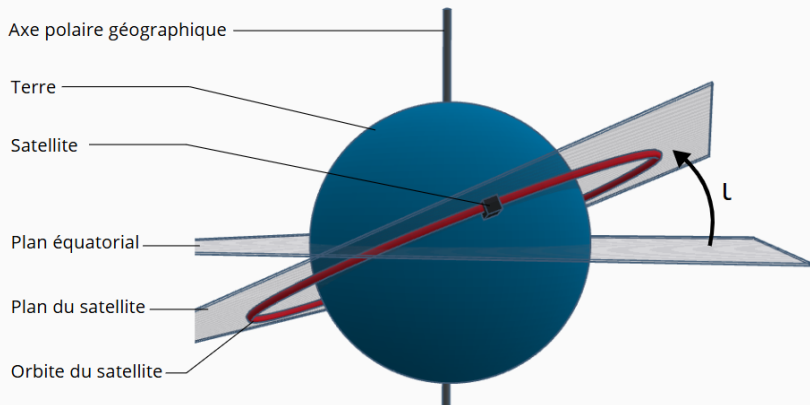


Figure 10 - orbite du satellite autour de la Terre et de l'inclinaison orbitale i

Trajectoire pendant la désaturation

Durée caractéristique de désaturation t^* : 5 min

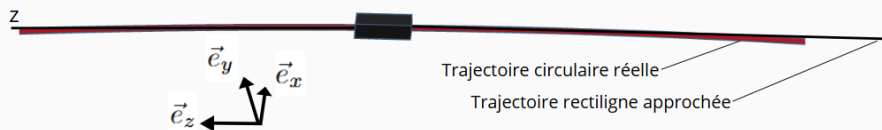


Figure 11 - approximation de la trajectoire du satellite sous la forme de droite

Modélisation du champ magnétique terrestre

$$R_T = 6400 \text{ km}$$

$$\text{alt} = 400 - 600 \text{ km}$$

$$R_N = 1000 \text{ km}$$

$$R_N \ll R_T + \text{alt}$$

$$\begin{cases} B_r(t) = 2 \frac{\mu_0 M}{4\pi r^3} \cos \theta(t) \\ B_\theta(t) = \frac{\mu_0 M}{4\pi r^3} \sin \theta(t) \\ B_\phi(t) = 0 \end{cases}$$

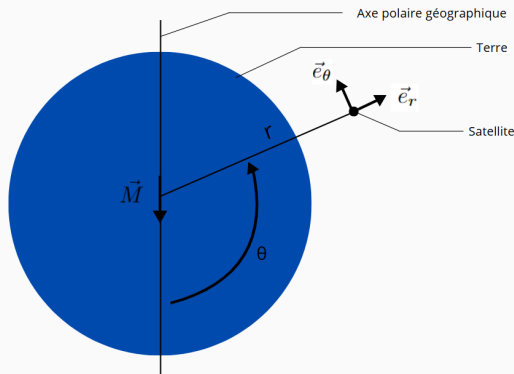


Figure 12 - Modélisation de la Terre comme dipôle magnétique dirigé selon l'axe polaire géographique

Variation de θ en fonction du temps

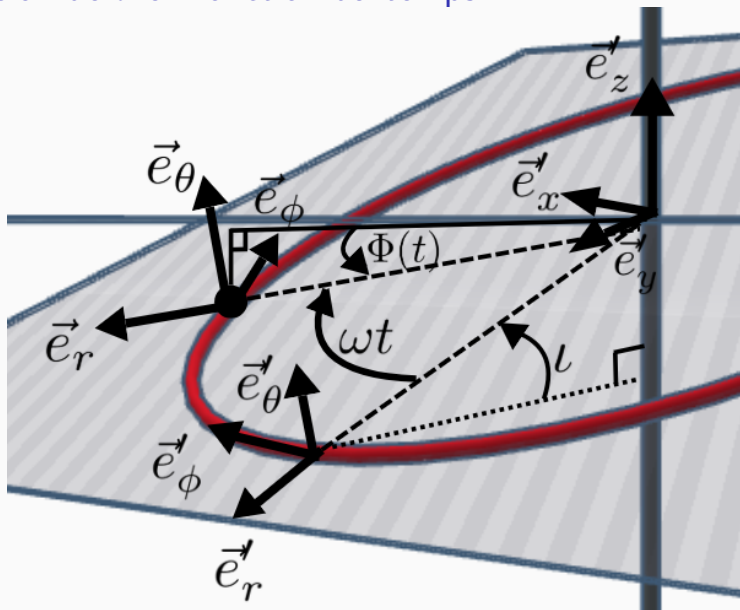


Figure 13 - position du satellite

Variation de θ en fonction du temps

$$\text{On a } \theta(t) = \frac{\pi}{2} + \Phi(t)$$

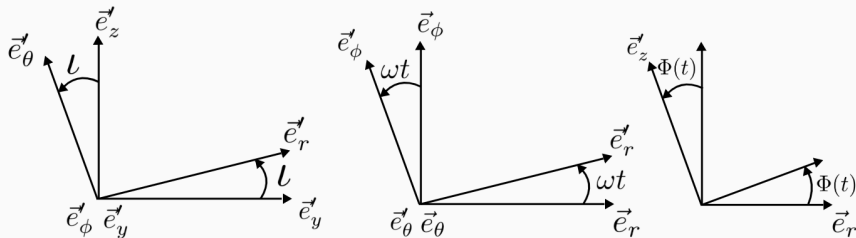


Figure 14 - figures de changement de base

$$-\sin(\Phi(t)) = \cos(\omega t) \sin(\iota) \vec{e}_z'$$

$$\theta(t) = \arccos(\cos(\omega t) \sin(\iota))$$

3 – Résolution du problème

Contraintes pour la désaturation

$$\text{Pour } \vec{B} = B_x \vec{e}_x + B_y \vec{e}_y + B_z \vec{e}_z$$

on a :

$$\vec{\Gamma}_x = M_x(B_y \vec{e}_z - B_z \vec{e}_y)$$

$$\vec{\Gamma}_y = M_y(B_z \vec{e}_x - B_x \vec{e}_z)$$

$$\vec{\Gamma}_z = M_z(B_x \vec{e}_y - B_y \vec{e}_x)$$

$$\vec{\Gamma}_s = -\vec{\Gamma}_r = -I_{rz} \dot{\omega}_r \cdot \vec{e}_z$$

Via le TMC, on en déduit la contrainte :

$$M_z = 0, \quad B_z = 0$$

Utilisation du champ terrestre

$$\begin{aligned}\vec{B}(t) &= B_r(t) \vec{e}_r + B_\theta(t) \vec{e}_\theta = B_r(t) \vec{e}_y + B_\theta(t) \vec{e}_x \\ &= \frac{\mu_0 M}{4\pi r^3} [2 \cos(\theta(t)) \vec{e}_y + \sin(\theta(t)) \vec{e}_x]\end{aligned}$$

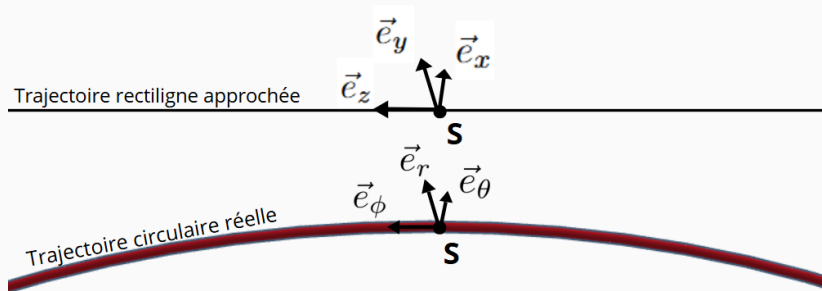


Figure 15 - visualisation des différentes bases

Formule finale

$$\Gamma_r(t) = B_y(t) M_y(t) - B_x(t) M_x(t)$$

$$\vec{B}(t) = \frac{\mu_0 M}{4\pi r^3} [2 \cos(\theta(t)) \vec{e}_y + \sin(\theta(t)) \vec{e}_x]$$

$$\theta(t) = \arccos(\cos(\omega t) \sin(\iota))$$

$$\Gamma_r(t) = I_{rz} \dot{\omega}_r(t)$$

$$\Gamma_r(t) = \frac{\mu_0 M}{4\pi r^3} \left(2 \cos(\omega t) \sin(\iota) \cdot M_y(t) - \sqrt{1 - [\cos(\omega t) \sin(\iota)]^2} \cdot M_x(t) \right)$$

4 – Application réelle

Exemple : module iADCS-200 pour CubeSat

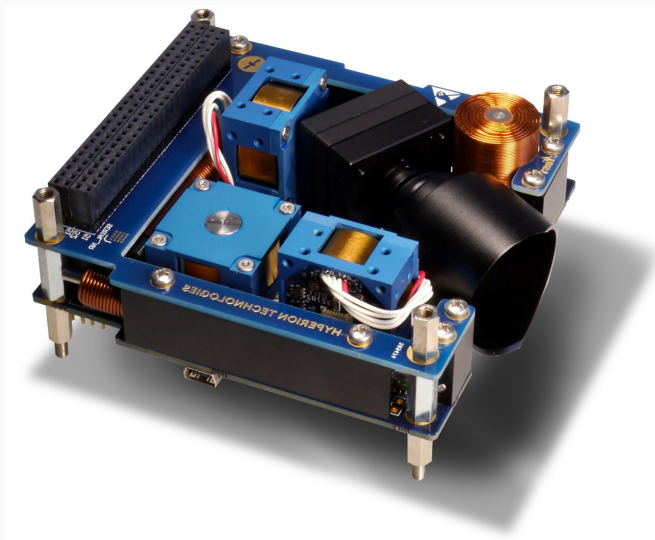


Figure 16 - Photo du module iADCS-200 ($90 \times 95 \times 32 \text{ mm}^3$)

Caractéristiques importantes

Magnetorquers

HT-MTQ200 Series

- 80x11x11 / 25x19x19 mm³
- Intended for small (up 4U) CubeSats
- Highly efficient
- Two models:
 - HT-MTQ200.20: 0.2 Am² , 100 mW, boost to 1 Am²
 - HT-MTQ200.15: 0.15 Am², 300 mW, boost to 0.25 Am²

Reaction Wheels

HT-RW200/RW210 Series

- 25x25x15 mm³
- Intended for small (up 4U) CubeSats
- Three models:
 - HT-RW200.15: 1.5 mNm, available now
 - HT-RW210.30: 3.0 mNm, available August 2015
 - HT-RW210.60: 6.0 mNm, available August 2015

TECHNICAL SPECIFICATIONS

Performance		
Total momentum storage	+/- 1.5, +/- 3.0, +/- 6.0	mN.m.s
Maximum torque	+/- 0.1	mN.m
Maximum rotation rate	10000 / 15000 / 15000	rpm
Control accuracy	+/- 0.5	rpm

Figure 17 - extrait de la fiche technique de HYPERION TECHNOLOGIES

Méthode d'Euler explicite

On crée une liste t d'instants espacés de Δt .
Pour tout k , on a :

$$I_{rz}\dot{\omega}_r(t[k]) = \Gamma_r(t[k])$$

$$\dot{\omega}_r(t[k]) = \frac{\omega_r(t[k]) - \omega_r(t[k-1])}{t[k] - t[k-1]}$$

$$\omega_r(t[k]) = \omega_r(t[k-1]) + \frac{\Delta t}{I_{rz}}\Gamma_r(t[k])$$

Méthode numérique

Pour tout k , on a :

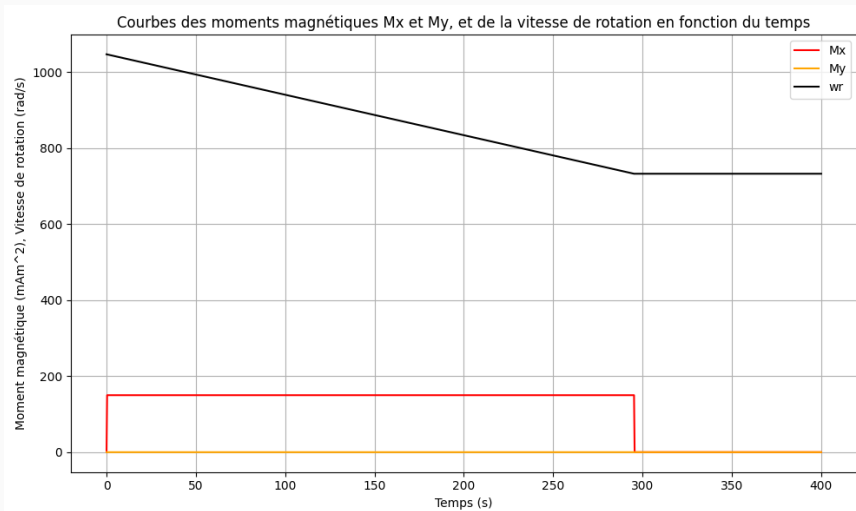
$$\Gamma_r(t[k]) = B_y(t[k]) M_y(t[k]) - B_x(t[k]) M_x(t[k])$$

avec :

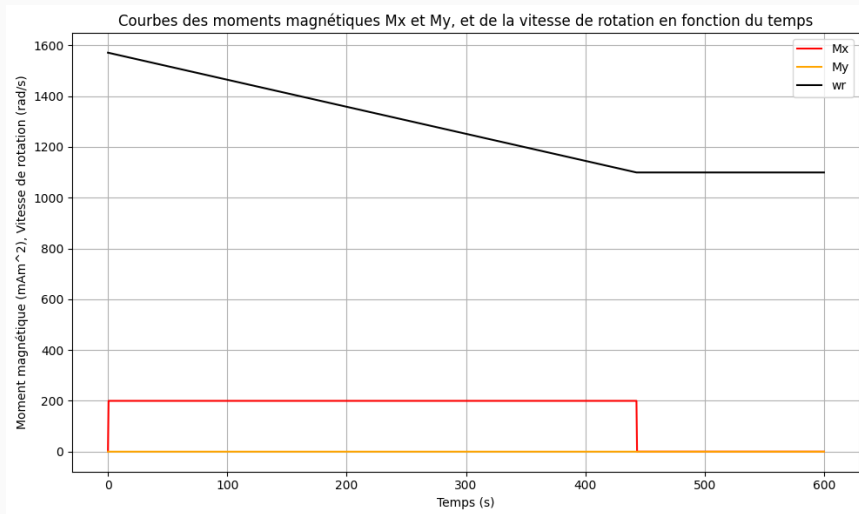
$$B_y(t[k]) = \frac{\mu_0 M}{4\pi r^3} \cdot 2 \cos(\omega t[k]) \sin(\iota)$$

$$B_x(t[k]) = -\frac{\mu_0 M}{4\pi r^3} \cdot \sqrt{1 - (\cos(\omega t[k]) \sin(\iota))^2}$$

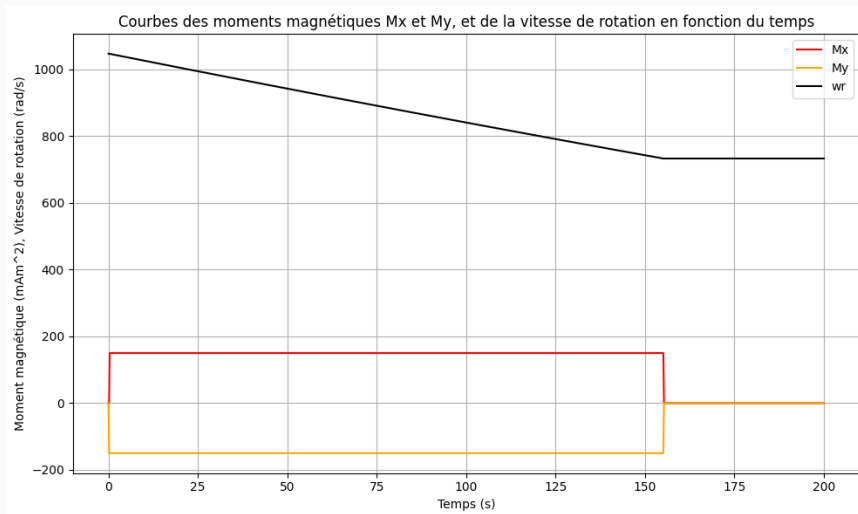
Résultats – Petit, 400km d'altitude, plan équatorial



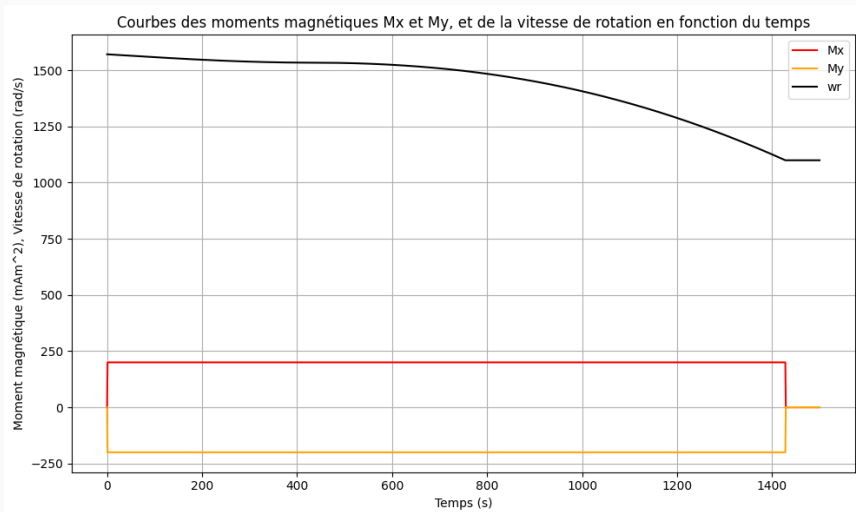
Résultats – Grand, 400km d'altitude, plan équatorial



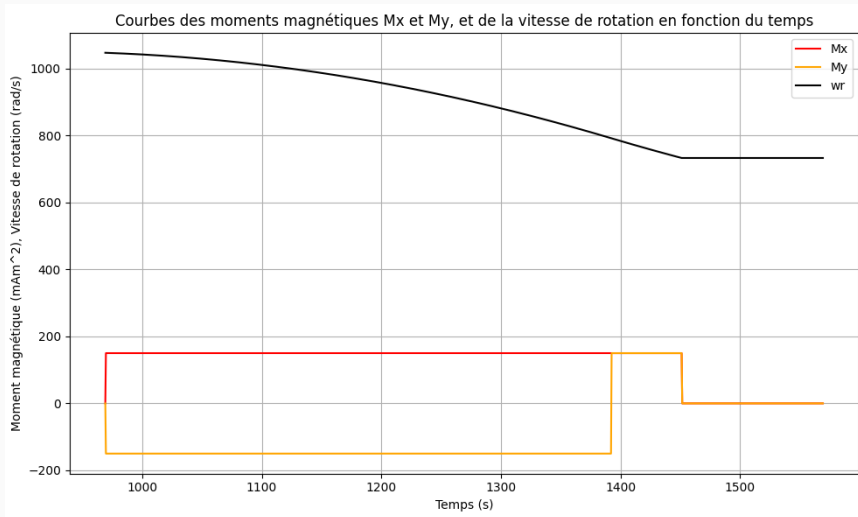
Résultats – Petit, alt = 400km, $t_0 = 0$, $\iota = \pi/2$



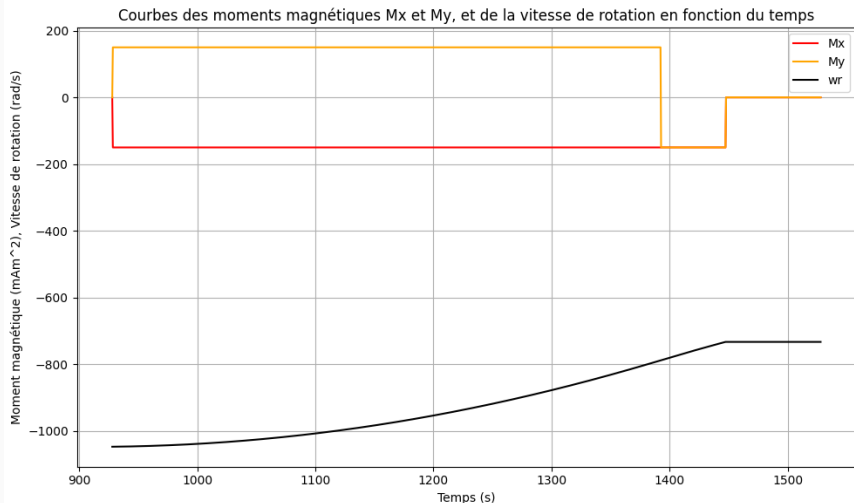
Résultats – Grand, alt = 600km, $t_0 = 0$, $\iota = \pi/6$



Résultats - Petit, alt = 600km, $t_0 = T/6$, $\iota = \pi/3$



Résultats - Petit, alt = 600km, $t_0 = T/6$, $\iota = \pi/3$,
 $\dot{\omega}_{sat} < 0$



Conclusion

Merci de votre attention

Annexe - Calcul contrainte pour la désaturation

$$\begin{cases} M_y B_z = M_z B_y \\ M_z B_x = M_x B_z \\ M_x B_y - M_y B_x = I_{rz} \dot{\omega}_r \\ \dot{\omega}_r \neq 0 \end{cases}$$

Supposons $B_z \neq 0$. Les deux premières équations donnent :

$$M_x = \frac{B_y}{B_z} M_z \text{ et } M_y = \frac{B_x}{B_z} M_z$$

Donc en injectant dans la 3ème équation :

$$0 = I_{rz} \dot{\omega}_r$$

Donc $B_z = 0$. Replacé dans les deux premières équations :

$$\begin{cases} M_z B_x = 0 \\ M_z B_y = 0 \end{cases} \Rightarrow M_z = 0$$

Conclusion : $M_z = 0$ et $B_z = 0$

Annexe - Code - constante

```
import matplotlib.pyplot as plt
import numpy as np

##constante
Masse_T = 6.0 * 10**24 #kg
R_T = 6400 #km
Moment_T = 6.4 * 10**22 #Am^2
mu_0 = 4*np.pi*10**-7 #H/m
G = 6.674*10**-11 #SI
p = 0.7 #SU, "ratio de désaturation"
masse_sat = 4 #kg, influence minime : considéré constant
Gammar_max = 1 * 10**-4 #Nm, jamais atteinte en pratique
```

##carac variable

```
#Magnéto-coupleur
Moment_petit = 0.15 #Am^2
Moment_grand = 0.2 #Am^2

#Roue de réaction
    #petit
Sto_mom_petit = 3 * 10**-3 #Nms
w_sat_petit = 2*np.pi/60*10000 #rad/s
    #grand
Sto_mom_grand = 6 * 10**-3 #Nms
w_sat_grand = 2*np.pi/60*15000 #rad/s
```

Annexe - Code - Temps

##Pour un système précis

```
t0 = T/6      #0→T (sec)
iota = np.pi/3  #0→pi/2 (rad)
alt = 400      #400→600 (km)

w_t0 = -w_sat_petit
w_fin = p * w_t0
Sto_mom = Sto_mom_petit
Mx_max, My_max = Moment_petit, Moment_petit
```

#calcul de certaines constantes

```
Irz = Sto_mom / np.abs(w_t0)
r = (alt + R_T) * 10**3
T = 2*np.pi*np.sqrt(r**3/(G*(Masse_T+masse_sat))) #s
omega = 2*np.pi/T
```

##Temps

```
durée = 600      #sec
nbpoint = 1200
delta_t = durée/nbpoint

t = np.linspace(t0, durée + t0, nbpoint)
```

Annexe - Code - Fonctions

```
##Fonction
#créer nos listes pour tracer les courbes
Gammar = [0]
Mx = [0]
My = [0]
wI = [w_t0]

#calcul de By et Bx à chaque instant

def By(instant) :
    return mu_0*Moment_T/(4*np.pi*r**3) * 2 * np.cos(omega*instant) * np.sin(iota)

def Bx(instant) :
    return mu_0*Moment_T/(4*np.pi*r**3) * np.sqrt(1 - (np.cos(omega*instant) * np.sin(iota))**2)

def couple_fournis_par_MC(By_k, Bx_k, Mx_k, My_k, signeBy_k, signeBx_k) :
    return np.abs(signedBy_k * By_k * My_k - signeBx_k * Bx_k * Mx_k)
```

Annexe - Code - Fonction

```
def calcul(t) :
    for k in range(nbpoint-1) :

        #calcul des termes à chaque instant
        By_k = By(t[k])
        Bx_k = Bx(t[k])
        signeBy_k = np.sign(By_k)
        signeBx_k = np.sign(Bx_k)
        signew = np.sign(w_t0)

        Couple_max_fournis_par_MC = couple_fournis_par_MC(By_k, Bx_k, Mx_max, My_max, signeBy_k, signeBx_k)

        if np.abs(wr[k]) ≥ np.abs(w_fin) :

            if np.abs(Couple_max_fournis_par_MC) ≤ np.abs(Gammar_max) :
                Gammar.append(- signew * Couple_max_fournis_par_MC)
                Mx.append(- (-1) * signew * signeBx_k * Mx_max)
                My.append(- signew * signeBy_k * My_max)

            else : #n'arrive jamais car le couple de la roue est largement > à celui des magnétocoupleurs
                None

            wr.append(wr[-1] + delta_t/Irz*Gammar[-1])

        else :
            Gammar.append(0)
            Mx.append(0)
            My.append(0)
            wr.append(wr[k])

calcul(t)
```

Annexe - Code - Affichage

```
Gammar = np.array(Gammar) #pas utiliser
Mx = 1000*np.array(Mx)    #mAm^2
My = 1000*np.array(My)    #mAm^2
wr = np.array(wr)         #rad/s

##graphe
plt.figure(figsize=(10, 6))
plt.plot(t, Mx, label='Mx', color='red')
plt.plot(t, My, label='My', color='orange')
plt.plot(t, wr, label='wr', color='black')
plt.title('Courbes des moments magnétiques Mx et My, et de la vitesse de rotation en fonction du temps')
plt.xlabel('Temps (s)')
plt.ylabel('Moment magnétique (mAm^2), Vitesse de rotation (rad/s)')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```